

Project 1 — The Grader

CS496 AI Engineering · Code Review Comment Quality · Fall 2026
Team: Skander Bedoui, Khalil, Yasmine, Abdallah Djaraya

Provisional report: the harness, judge reliability analysis, bias tests, and system runs are not present. Pending fields below must be replaced with timestamped repository evidence before submission.

1. Task and motivation

The system receives a code snippet and produces exactly one review comment. A good comment is **correct**—it identifies a real issue or correctly reports that none is present—and **useful**—specific, actionable where appropriate, and professional. Exact matching cannot grade this task because multiple phrasings can describe the same defect, while lexical similarity can reward a fluent but technically false comment. The project therefore combines human labels with a structured LLM judge.

2. Golden dataset and labelling

The checked-in JSONL contains 160 examples across Python (110) and JavaScript (50), split into 112 development and 48 reserved test items. Every row contains two human labels. The labelling guide defines good, bad, and borderline cases with six examples.

Measure	Repository evidence
Items	160
Dev / test	112 / 48
Human agreement	86.25%
Cohen's kappa	0.72

Source: data/golden_set.jsonl, data/labelling_guide.md, and data/agreement_report.md. Dropped-item count/methodology requires team confirmation.

3. Harness and judge

The intended pipeline is code → system → structured comment → judge → per-item JSONL. The system schema is {"comment": "string"}; the documented judge schema contains verdict, correctness, usefulness, and a short reason. Each result must also preserve separate system/judge token counts and latency.

Status: harness implementation and results are pending Khalil. Judge-vs-human agreement, position-order swaps, verbosity-padding tests, and concrete failure examples are pending Yasmine. No reliability claim is currently supportable.

4. System prompt engineering

Version 1 is intentionally a baseline: it requests one professional comment about the most important demonstrable issue, rejects unsupported assumptions, permits a no-issue answer, and requires schema-valid JSON. The harness should load the file as the system message and pass code separately as the user message.

Version	Development items	Good	Correct	Useful	Mean cost / latency
v1 baseline	Pending harness and judge integration				

No v2/v3 was created: without actual development failures, additional versions would be artificial. After v1 runs, failures will be classified as false positives, missed bugs, vague comments, unsupported claims, clean-code errors, or schema violations. The selected prompt will be frozen before one authorized test evaluation.

5. Cost and scalability

For each call, cost equals (input tokens × input price + output tokens × output price) / 1,000,000. The tool calculates system and judge costs separately because models and prices may differ, then

reports total and mean cost, cost per 1,000 items, daily cost, and 100× daily-volume cost. Prices are user inputs; no provider price is fabricated.

Measured cost: pending result JSONL. Linear scaling is only a baseline: batching and prompt caching may reduce effective cost, retries add calls, concurrency changes throughput, and provider rate limits may prevent the projected volume.

6. Limitations and conclusion

- System, judge, bias, cost, and latency measurements remain pending integration.
- The report must not be submitted as final until those results replace every pending field.
- The current work establishes an auditable baseline prompt, validated cost formulas, explicit interfaces, and honest documentation without using the reserved test labels for optimization.

The final conclusion will compare measured prompt versions, judge reliability, bias rates, and the single frozen-prompt test result.